Unisoc Confidential For longcheer

Android Multimedia AudioFW 音量调节流程

文档版本	V1.0
发布日期	2023-07-05

紫光展锐（上海）科技有限公司

版权所有 © 紫光展锐（上海）科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐（上海）科技有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负责任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

商标声明

紫光展锐、UNISOC、、展讯、Spreadtrum、SPRD、锐迪科、RDA 及其他紫光展锐的商标均为紫光展锐（上海）科技有限公司及/或其子公司、关联公司所有。

Bluetooth®文字商标和徽标为蓝牙技术联盟的注册商标，紫光展锐（上海）科技有限公司对此类商标的任何使用均已获得许可。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

免责声明

本文档可能包含第三方内容，包括但不限于第三方信息、软件、组件、数据等。紫光展锐不控制且不对第三方内容承担任何责任，包括但不限于准确性、兼容性、可靠性、可用性、合法性、适当性、性能、不侵权、更新状态等，除非本文档另有明确说明。在本文档中提及或引用任何第三方内容不代表紫光展锐对第三方内容的认可、承诺或保证。

用户有义务结合自身情况，检查上述第三方内容的可用性。若需要第三方许可，应通过合法途径获取第三方许可，除非本文档另有明确说明。

Unisoc Confidential For longcheer

紫光展锐（上海）科技有限公司

前 言

概述

本文档基于 Android T 详细地介绍了 audio fw 部分音量调节的过程

读者对象

本文档主要适用于 audio fw 相关的研发人员。读者人员必须具备以下经验和技能：

- 具有 audio 相关的基础。
- 熟悉 audioservice audiopolicy audioflinger。

缩略语

缩略语	英文全称	中文解释

Unisoc Confidential For longcheer

符号约定

在本文中可能出现下列符号，每种符号的说明如下。

符号	说明
说明	用于突出重要或关键信息、补充信息和小窍门等。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害。
注意	用于突出容易出错的操作。 “注意”不是安全警示信息，不涉及人身、设备及环境伤害。
警告	用于可能无法恢复的失误操作。 “警告”不是危险警示信息，不涉及人身及环境伤害。

 版权所有 © 紫光展锐（上海）科技有限公司

变更信息

文档版本	发布日期	修改说明
V1.0	2023-07-05	• 第一次正式发布

关键字

Audio 音量调节

Unisoc Confidential For longcheer

目 录

1 音频流 1
2 JAVA 层输出设备定义 3
3 默认音量 5
3.1 最大音量 5
3.2 最小音量 5
3.3 默认初始音量 6
4 JAVA 层音量调节 7
4.1 onKeyDown 7
4.2 dispatchVolumeKeyEventToSessionAsSystemService 8
4.3 dispatchVolumeKeyEventAsSystemService 10
4.4 adjustSuggestedStreamVolumeForUid 13
4.5 applyDeviceVolume_syncVSS 19
4.6 setStreamVolumeIndexAS 19
5 Native 层音量调节 21
5.1 JNI 中的 setStreamVolumeIndex 21
5.2 setStreamVolumeIndex 22
5.3 checkAndSetVolume 23
5.4 outputDesc->setVolume 24
5.5 AudioPolicyClient::setVoiceVolume 27
A 附录（Appendix heading 1） 29
A.1 代码（Appendix heading 2） 29
A.2 其它参考信息 29

Unisoc Confidential For longcheer

1 音频流

在 android 系统中，音量都是分开控制的，比如媒体音量，铃声音量，闹钟音量，通话音量等，各种音量都是通过音频流来区别不同声音类型的。在 Android T 里面一共定义了 11 种流类型。每种流类型都定义了最大音量(MAX_STREAM_VOLUME)，最小音量（MIN_STREAM_VOLUME），以及默认音量（DEFAULT_STREAM_VOLUME），虽然定义了 11 种流类型，但是实际我们能够调节的音量类型也就 5 种，所以还存在一个流的映射关系，然后这个流就可以使用另一个流的音量了

音频流类型：

frameworks/base/media/java/android/media/AudioSystem.java

```
80      /** @hide Used to identify the default audio stream volume */
81      @TestApi
82      public static final int STREAM_DEFAULT = -1; //默认类型
83      /** @hide Used to identify the volume of audio streams for phone calls */
84      public static final int STREAM_VOICE_CALL = 0; //通话声音
85      /** @hide Used to identify the volume of audio streams for system sounds */
86      public static final int STREAM_SYSTEM = 1; //系统声音
87      /** @hide Used to identify the volume of audio streams for the phone ring and
message alerts */
88      public static final int STREAM_RING = 2; //来电铃声
89      /** @hide Used to identify the volume of audio streams for music playback */
90      public static final int STREAM_MUSIC = 3; //媒体音
91      /** @hide Used to identify the volume of audio streams for alarms */
92      public static final int STREAM_ALARM = 4; //闹钟
93      /** @hide Used to identify the volume of audio streams for notifications */
94      public static final int STREAM_NOTIFICATION = 5; //通知
95      /** @hide
96       * Used to identify the volume of audio streams for phone calls when connected on
bluetooth */
97      public static final int STREAM_BLUETOOTH_SCO = 6; //蓝牙通话
98      /** @hide Used to identify the volume of audio streams for enforced system sounds
in certain
99       * countries (e.g camera in Japan) */
100      @UnsupportedAppUsage(maxTargetSdk = Build.VERSION_CODES.R, trackingBug =
170729553)
101      public static final int STREAM_SYSTEM_ENFORCED = 7; //强制输出，拍照音
102      /** @hide Used to identify the volume of audio streams for DTMF tones */
103      public static final int STREAM_DTMF = 8; //Tone 音，如按键音
104      /** @hide Used to identify the volume of audio streams exclusively transmitted
through the
105       * speaker (TTS) of the device */
106      public static final int STREAM_TTS = 9; //文字转语音流
```

Unisoc Confidential For longcheer

 版权所有 © 紫光展锐（上海）科技有限公司

```
107     /** @hide Used to identify the volume of audio streams for accessibility prompts
*/
108     public static final int STREAM_ACCESSIBILITY = 10; //辅助功能提示
109     /** @hide Used to identify the volume of audio streams for virtual assistant */
110     public static final int STREAM_ASSISTANT = 11; //语音助手
```

Unisoc Confidential For longcheer

2 JAVA 层输出设备定义

JAVA 层中对声音输入输出设备的定义在 AudioSystem.java 中，下面列举一些常用的输出设备定义

这里写明了必须和 Native 层的设备定义保持同步，即 system/core/audio.h 中的设备定义

frameworks/base/media/java/android/media/AudioSystem.java

```
939         // audio device definitions: must be kept in sync with values in
system/core/audio.h
940         //
941         /** @hide */
942         public static final int DEVICE_NONE = 0x0;
943         // reserved bits
944         /** @hide */
945         public static final int DEVICE_BIT_IN = 0x80000000;
946         /** @hide */
947         public static final int DEVICE_BIT_DEFAULT = 0x40000000;
948         // output devices, be sure to update AudioManager.java also
949         /** @hide */
950         @UnsupportedAppUsage
951         public static final int DEVICE_OUT_EARPIECE = 0x1;//听筒
952         /** @hide */
953         @UnsupportedAppUsage
954         public static final int DEVICE_OUT_SPEAKER = 0x2;//扬声器
955         /** @hide */
956         @UnsupportedAppUsage
957         public static final int DEVICE_OUT_WIRED_HEADSET = 0x4;//线控耳机
958         /** @hide */
959         @UnsupportedAppUsage
960         public static final int DEVICE_OUT_WIRED_HEADPHONE = 0x8;//普通耳机，不能操控播放
961         /** @hide */
962         @UnsupportedAppUsage
963         public static final int DEVICE_OUT_BLUETOOTH_SCO = 0x10;//单声道蓝牙耳机
964         /** @hide */
965         @UnsupportedAppUsage
966         public static final int DEVICE_OUT_BLUETOOTH_SCO_HEADSET = 0x20;//蓝牙耳机，带话
筒
967         /** @hide */
968         @UnsupportedAppUsage
969         public static final int DEVICE_OUT_BLUETOOTH_SCO_CARKIT = 0x40;//车载免提蓝牙设备
970         /** @hide */
971         @UnsupportedAppUsage
```

Unisoc Confidential For longcheer

 版权所有 © 紫光展锐（上海）科技有限公司

```
972     public static final int DEVICE_OUT_BLUETOOTH_A2DP = 0x80;//蓝牙立体声，音乐播放
973     /** @hide */
974     @UnsupportedAppUsage
975     public static final int DEVICE_OUT_BLUETOOTH_A2DP_HEADPHONES = 0x100;蓝牙立体耳机
976     /** @hide */
991     @UnsupportedAppUsage
992     public static final int DEVICE_OUT_USB_ACCESSORY = 0x2000;//usb 配件
993     /** @hide */
994     @UnsupportedAppUsage
995     public static final int DEVICE_OUT_USB_DEVICE = 0x4000;usb 设备
996     /** @hide */
997     @UnsupportedAppUsage
998     public static final int DEVICE_OUT_REMOTE_SUBMIX = 0x8000;
1023     /** @hide */
1024     public static final int DEVICE_OUT_USB_HEADSET = 0x4000000;//usb 耳机
1036     /** @hide */
1037     public static final int DEVICE_OUT_DEFAULT = DEVICE_BIT_DEFAULT;//默认设备
```

Unisoc Confidential For longcheer

 版权所有 © 紫光展锐（上海）科技有限公司

3 默认音量

3.1 最大音量

frameworks/base/services/core/java/com/android/server/audio/AudioService.java

```
    protected static int[] MAX_STREAM_VOLUME = new int[] {
        5,  // STREAM_VOICE_CALL
        7,  // STREAM_SYSTEM
        7,  // STREAM_RING
        15, // STREAM_MUSIC
        7,  // STREAM_ALARM
        7,  // STREAM_NOTIFICATION
        15, // STREAM_BLUETOOTH_SCO
        7,  // STREAM_SYSTEM_ENFORCED
        15, // STREAM_DTMF
        15, // STREAM_TTS
        15, // STREAM_ACCESSIBILITY
        15  // STREAM_ASSISTANT
    };
```

Unisoc Confidential For longcheer

3.2 最小音量

frameworks/base/services/core/java/com/android/server/audio/AudioService.java

```
    protected static int[] MIN_STREAM_VOLUME = new int[] {
        1,  // STREAM_VOICE_CALL
        0,  // STREAM_SYSTEM
        0,  // STREAM_RING
        0,  // STREAM_MUSIC
        1,  // STREAM_ALARM
        0,  // STREAM_NOTIFICATION
        0,  // STREAM_BLUETOOTH_SCO
        0,  // STREAM_SYSTEM_ENFORCED
        0,  // STREAM_DTMF
        0,  // STREAM_TTS
        1,  // STREAM_ACCESSIBILITY
        0   // STREAM_ASSISTANT
    };
```

 版权所有 © 紫光展锐（上海）科技有限公司

3.3 默认初始音量

frameworks/base/media/java/android/media/AudioSystem.java

```
2289        public static int[] DEFAULT_STREAM_VOLUME = new int[] {
2290            4,  // STREAM_VOICE_CALL
2291            7,  // STREAM_SYSTEM
2292            5,  // STREAM_RING
2293            5, // STREAM_MUSIC
2294            6,  // STREAM_ALARM
2295            5,  // STREAM_NOTIFICATION
2296            7,  // STREAM_BLUETOOTH_SCO
2297            7,  // STREAM_SYSTEM_ENFORCED
2298            5, // STREAM_DTMF
2299            5, // STREAM_TTS
2300            5, // STREAM_ACCESSIBILITY
2301            5, // STREAM_ASSISTANT
2302        };
```

音频流映射在 android 系统中不同设备的映射不尽相同，一共定义了三种设备的音频流的映射，分别是 STREAM_VOLUME_ALIAS_VOICE 具有语音功能的设备，STREAM_VOLUME_ALIAS_TELEVISION 电视或者机顶盒，STREAM_VOLUME_ALIAS_DEFAULT 平板之类的设备，如下是 STREAM_VOLUME_ALIAS_VOICE 的流类型的映射关系

frameworks/base/services/core/java/com/android/server/audio/AudioService.java

Unisoc Confidential For longcheer

```
427         private final int[] STREAM_VOLUME_ALIAS_VOICE = new int[] {
428             AudioSystem.STREAM_VOICE_CALL,          // STREAM_VOICE_CALL
429             AudioSystem.STREAM_RING,                // STREAM_SYSTEM
430             AudioSystem.STREAM_RING,                // STREAM_RING
431             AudioSystem.STREAM_MUSIC,               // STREAM_MUSIC
432             AudioSystem.STREAM_ALARM,               // STREAM_ALARM
433             AudioSystem.STREAM_RING,                // STREAM_NOTIFICATION
434             AudioSystem.STREAM_BLUETOOTH_SCO,       // STREAM_BLUETOOTH_SCO
435             AudioSystem.STREAM_RING,                // STREAM_SYSTEM_ENFORCED
436             AudioSystem.STREAM_RING,                // STREAM_DTMF
437             AudioSystem.STREAM_MUSIC,               // STREAM_TTS
438             AudioSystem.STREAM_MUSIC,               // STREAM_ACCESSIBILITY
439             AudioSystem.STREAM_MUSIC                // STREAM_ASSISTANT
440         };
```

 版权所有 © 紫光展锐（上海）科技有限公司

4 JAVA 层音量调节

4.1 onKeyDown

frameworks/base/core/java/com/android/internal/policy/PhoneWindow.java

>>>按键事件

```
1918        protected boolean onKeyDown(int featureId, int keyCode, KeyEvent event) {
...
1939            switch (keyCode) {
1940                case KeyEvent.KEYCODE_VOLUME_UP:
1941                case KeyEvent.KEYCODE_VOLUME_DOWN:
1942                case KeyEvent.KEYCODE_VOLUME_MUTE: {
1943                    // If we have a session send it the volume command, otherwise
1944                    // use the suggested stream.
1945                    if (mMediaController != null) {
```

mMediaController 不为空，调用 dispatchVolumeKeyEventToSessionAsSystemService 方法，当前测试发现，调节音量时处在 YTmusic 界面时，mMediaController 不为空，执行此方法，其他应用场景还没发现设置了 mMediaController

Unisoc Confidential For longcheer

```
1946                        getMediaSessionManager().dispatchVolumeKeyEventToSessionAsSystem
Service(event,
1947                                mMediaController.getSessionToken());
1948                    } else {
```

mMediaController 为空，调用 dispatchVolumeKeyEventAsSystemService，luncher 界面，设置界面，qq 音乐界面等界面调节音量都是走的这里，这里的 stream 是 mVolumeControlStreamType 默认值是-2147483648

```
1949                        getMediaSessionManager().dispatchVolumeKeyEventAsSystemService(e
vent,
1950                                mVolumeControlStreamType);
1951                    }
1952                    return true;
```

说明

PhoneWindow.java 的 onKeyDown 方法就是接收到手机的音量按键事件，然后根据是否设置了 MediaController 分别走不同的两个方法

 版权所有 © 紫光展锐（上海）科技有限公司

4.2 dispatchVolumeKeyEventToSessionAsSystemService

首先看下设置了 MediaController 的情况下走的 dispatchVolumeKeyEventToSessionAsSystemService 方法

frameworks/base/media/java/android/media/session/MediaSessionManager.java

```
    public void dispatchVolumeKeyEventToSessionAsSystemService(@NonNull KeyEvent keyEvent,
            @NonNull MediaSession.Token sessionToken) {
        Objects.requireNonNull(sessionToken, "sessionToken shouldn't be null");
        Objects.requireNonNull(keyEvent, "keyEvent shouldn't be null");
        try {
```

继续调用 MediaSessionService.java 中的 dispatchVolumeKeyEventToSessionAsSystemService 方法

```
            mService.dispatchVolumeKeyEventToSessionAsSystemService(mContext.getPackageName(),
                    mContext.getOpPackageName(), keyEvent, sessionToken);
```

>>> MediaSessionService 的 dispatchVolumeKeyEventToSessionAsSystemService

Unisoc Confidential For longcheer

frameworks/base/services/core/java/com/android/server/media/MediaSessionService.java

```
    @Override
    public void dispatchVolumeKeyEventToSessionAsSystemService(String packageName,
            String opPackageName, KeyEvent keyEvent, MediaSession.Token sessionToken) {
```

...

这里进行 direction 赋值

KeyEvent.ACTION_DOWN 按下按键，根据按键的 KeyCode 设置音量调整的方向，调大音量，调小音量，静音操作对应的 direction 分别是 0 不调整音量，1 调大音量，-1 调小音量，101 静音操作，如果当前没有静音则静音，当前已经静音就解除静音

```
            switch (keyEvent.getAction()) {
                case KeyEvent.ACTION_DOWN: {
                    int direction = 0;
                    switch (keyEvent.getKeyCode()) {
                        case KeyEvent.KEYCODE_VOLUME_UP:
                            direction = AudioManager.ADJUST_RAISE;
                            break;
                        case KeyEvent.KEYCODE_VOLUME_DOWN:
                            direction = AudioManager.ADJUST_LOWER;
                            break;
                        case KeyEvent.KEYCODE_VOLUME_MUTE:
```

 版权所有 © 紫光展锐（上海）科技有限公司

```
                                    direction = AudioManager.ADJUST_TOGGLE_MUTE;
                                    break;
                                }
...
```

继续调用 record.adjustVolume

```
                                record.adjustVolume(packageName, opPackageName, pid, uid,
                                    true /* asSystemService */, direction,
                                    AudioManager.FLAG_SHOW_UI, false /* useSuggested */);
                                break;
                            }
```

>>> MediaSessionRecord 的 adjustVolume 方法

frameworks/base/services/core/java/com/android/server/media/MediaSessionRecord.java

```
    public void adjustVolume(String packageName, String opPackageName, int pid, int uid,
            boolean asSystemService, int direction, int flags, boolean useSuggested) {
```

这里根据 mAudioAttrs 获取对应的 stream

```
        int stream = getVolumeStream(mAudioAttrs);
```

调用 postAdjustLocalVolume 方法

```
        postAdjustLocalVolume (stream, direction, flags, opPackageName, pid, uid,
                asSystemService, useSuggested, previousFlagPlaySound);
```

>>>postAdjustLocalVolume 方法

```
    private void postAdjustLocalVolume(final int stream, final int direction, final int flags,
            final String callingOpPackageName, final int callingPid, final int callingUid,
            final boolean asSystemService, final boolean useSuggested,
            final int previousFlagPlaySound) {
...
        mHandler.post(new Runnable() {
            @Override
            public void run() {
                try {
                    if (useSuggested) {
...
                    } else {
```

Unisoc Confidential For longcheer

子线程调用 mAudioManager. adjustStreamVolumeForUid 方法调整音量

```
572                         mAudioManager.adjustStreamVolumeForUid(stream, direction,
flags,
573                                 opPackageName, uid, pid,
574                                 mContext.getApplicationInfo().targetSdkVersion);
575                     }
```

说明

至此调用到 AudioManager 的 adjustStreamVolumeForUid 方法，传入的参数注意有流类型 stream，调节音量的方向 direction，以及一个标志 flags

说明

后面再介绍 AudioManager 部分的内容，下面看另一个方法 dispatchVolumeKeyEventAsSystemService

4.3 dispatchVolumeKeyEventAsSystemService

frameworks/base/media/java/android/media/session/MediaSessionManager.java

```
683     public void dispatchVolumeKeyEventAsSystemService(@NonNull KeyEvent keyEvent, int
streamType) {
684         dispatchVolumeKeyEventInternal(keyEvent, streamType, /*musicOnly=*/false,
685             /*asSystemService=*/true);
686     }
```

Unisoc Confidential For longcheer

继续向下调用

```
688     private void dispatchVolumeKeyEventInternal(@NonNull KeyEvent keyEvent, int
stream,
689             boolean musicOnly, boolean asSystemService) {
690         Objects.requireNonNull(keyEvent, "keyEvent shouldn't be null");
691         try {
692             mService.dispatchVolumeKeyEvent(mContext.getPackageName(),
mContext.getOpPackageName(),
693                     asSystemService, keyEvent, stream, musicOnly);
694         } catch (RemoteException e) {
695             Log.e(TAG, "Failed to send volume key event.", e);
696         }
697     }
```

>>>调用 MediaSessionService 中的 dispatchVolumeKeyEvent 方法

frameworks/base/services/core/java/com/android/server/media/MediaSessionService.java

 版权所有 © 紫光展锐（上海）科技有限公司

```
1809            public void dispatchVolumeKeyEvent(String packageName, String opPackageName,
1810                    boolean asSystemService, KeyEvent keyEvent, int stream, boolean
musicOnly) {
…
1835                    } else {
1836                        // TODO: Consider the case when both volume up and down keys
are pressed
1837                        //       at the same time.
1838                        mVolumeKeyEventHandler.handleVolumeKeyEventLocked(packageNam
e, pid, uid,
1839                                asSystemService, keyEvent, opPackageName, stream,
musicOnly);
1840                    }
1841                }
```

>>>继续向下调用 handleVolumeKeyEventLocked

```
2558            void handleVolumeKeyEventLocked(String packageName, int pid, int uid,
2559                    boolean asSystemService, KeyEvent keyEvent, String
opPackageName, int stream,
2560                    boolean musicOnly) {
2561                handleKeyEventLocked(packageName, pid, uid, asSystemService,
keyEvent, false,
2562                        opPackageName, stream, musicOnly);
2563            }
```

Unisoc Confidential For longcheer

>>>继续向下调用 handleKeyEventLocked

```
2565            void handleKeyEventLocked(String packageName, int pid, int uid,
2566                    boolean asSystemService, KeyEvent keyEvent, boolean
needWakeLock,
2567                    String opPackageName, int stream, boolean musicOnly) {

…
2580                if (!needTracking(keyEvent, overriddenKeyEvents)) {
2581                    if (mKeyType == KEY_TYPE_VOLUME) {
```

手机按键音量调节走这里的 dispatchVolumeKeyEventLocked 方法

```
2582                        dispatchVolumeKeyEventLocked(packageName, opPackageName,
pid, uid,
2583                                asSystemService, keyEvent, stream, musicOnly);
```

>>>继续向下调用 dispatchVolumeKeyEventLocked

 版权所有 © 紫光展锐（上海）科技有限公司

```
1847            private void dispatchVolumeKeyEventLocked(String packageName, String
opPackageName, int pid,
1848                    int uid, boolean asSystemService, KeyEvent keyEvent, int stream,
1849                    boolean musicOnly) {
```

…

这里对调整的方向 direction 进行赋值

```
1854                switch (keyEvent.getKeyCode()) {
1855                    case KeyEvent.KEYCODE_VOLUME_UP:
1856                        direction = AudioManager.ADJUST_RAISE;
1857                        break;
1858                    case KeyEvent.KEYCODE_VOLUME_DOWN:
1859                        direction = AudioManager.ADJUST_LOWER;
1860                        break;
1861                    case KeyEvent.KEYCODE_VOLUME_MUTE:
1862                        isMute = true;
1863                        break;
1864                }
1865                if (down || up) {
```

这里对 flags 进行赋值

```
1866                    int flags = AudioManager.FLAG_FROM_KEY;
1867                    if (!musicOnly) {
1868                        // These flags are consistent with the home screen
1869                        if (up) {
1870                            flags |= AudioManager.FLAG_PLAY_SOUND |
AudioManager.FLAG_VIBRATE;
1871                        } else {
1872                            flags |= AudioManager.FLAG_SHOW_UI |
AudioManager.FLAG_VIBRATE;
1873                        }
1874                    }
```

```
1875                    if (direction != 0) {
1876                        // If this is action up we want to send a beep for non-music
events
1877                        if (up) {
```

如果是松开按键事件，direction 赋值为 0，不调整音量

```
1878                            direction = 0;
1879                        }
1880                        dispatchAdjustVolumeLocked(packageName, opPackageName, pid, uid,
1881                                asSystemService, stream, direction, flags, musicOnly);
1890            }
```

Unisoc Confidential For longcheer

>>>继续调用 dispatchAdjustVolumeLocked 方法

```
2199            private void dispatchAdjustVolumeLocked(String packageName, String
opPackageName, int pid,
2200                    int uid, boolean asSystemService, int suggestedStream, int
direction, int flags,
2201                    boolean musicOnly) {
```

…

这里通过子线程调用了 AudioManager 的 adjustSuggestedStreamVolumeForUid 方法

```
2229                    mHandler.post(new Runnable() {
2230                        @Override
2231                        public void run() {
```

…

```
2244                            try {
2245                                mAudioManager.adjustSuggestedStreamVolumeForUid(suggeste
dStream,
2246                                        direction, flags, callingOpPackageName,
callingUid, callingPid,
2247                                        getContext().getApplicationInfo().targetSdkVersi
on);
```

说明

至此也是调用到了 AudioManager 的 adjustSuggestedStreamVolumeForUid 方法

Unisoc Confidential For longcheer

注意

从以上可以看出，dispatchVolumeKeyEventToSessionAsSystemService 和 dispatchVolumeKeyEventAsSystemService 最终都是调用 AudioManager 的 adjustSuggestedStreamVolumeForUid 方法进行调整音量，区别就是 mediasession 部分的处理流程不一样

说明

接下来继续分析 AudioManager 的 adjustSuggestedStreamVolumeForUid 方法

4.4 adjustSuggestedStreamVolumeForUid

frameworks/base/media/java/android/media/AudioManager.java

```
7509        @SystemApi(client = SystemApi.Client.MODULE_LIBRARIES)
7510        public void adjustSuggestedStreamVolumeForUid(int suggestedStreamType, int
direction, int flags,
7511                @NonNull String packageName, int uid, int pid, int targetSdkVersion) {
7512            try {
```

```
7513                getService().adjustSuggestedStreamVolumeForUid(suggestedStreamType,
direction, flags,
7514                        packageName, uid, pid, UserHandle.getUserHandleForUid(uid),
targetSdkVersion);
```

>>>继续调用 AudioService 的 adjustSuggestedStreamVolumeForUid

frameworks/base/services/core/java/com/android/server/audio/AudioService.java

```
5314        /** @see AudioManager#adjustSuggestedStreamVolumeForUid(int, int, int, String,
int, int, int) */
5315        @Override
5316        public void adjustSuggestedStreamVolumeForUid(int streamType, int direction, int
flags,
…
5323            // direction and stream type swap here because the public
5324            // adjustSuggested has a different order than the other methods.
5325            adjustSuggestedStreamVolume(direction, streamType, flags, packageName,
packageName,
5326                    uid, pid, hasAudioSettingsPermission(uid, pid),
5327                    AudioDeviceVolumeManager.ADJUST_MODE_NORMAL);
5328        }
```

>>>继续调用 adjustSuggestedStreamVolume

```
2919    /** All callers come from platform apps/system server, so no attribution tag is needed */
2920    private void adjustSuggestedStreamVolume(int direction, int suggestedStreamType, int flags,
2921            String callingPackage, String caller, int uid, int pid, boolean hasModifyAudioSettings,
2922            int keyEventMode) {
…
2950            final int streamType;
2951            synchronized (mForceControlStreamLock) {
…
```

获取需要调整的音频流类型

getActiveStreamType 方法里面就是判断当前活跃的声音是哪个音频流类型就返回对应的 streamtype，默认是 music 流

```
2956                    final int maybeActiveStreamType =
getActiveStreamType(suggestedStreamType);
…
```

继续调用 adjustStreamVolume

Unisoc Confidential For longcheer

```
2994            adjustStreamVolume(streamType, direction, flags, callingPackage, caller,
uid, pid,
2995                    null, hasModifyAudioSettings, keyEventMode);
2996        }
```

>>>adjustStreamVolume 方法

```
3036        protected void adjustStreamVolume(int streamType, int direction, int flags,
3037                String callingPackage, String caller, int uid, int pid, String
attributionTag,
3038                boolean hasModifyAudioSettings, int keyEventMode) {
```

…

获取需要调整音量的设备

getDeviceForStream 方法最终是调用的 audiopolicymanager 的 **getDevicesForAttributes** 方法获取到的设备，这里不再详细介绍

```
3084            final int device = getDeviceForStream(streamTypeAlias);
```

计算调整步幅

```
3131                // convert one UI step (+/-1) into a number of internal units on the
stream alias
3132                step = rescaleStep(10, streamType, streamTypeAlias);
3133            }
3134
```

Unisoc Confidential For longcheer

这里关注下 streamState.adjustIndex 方法

注意

streamState.adjustIndex 方法调整的音量值将音量值保存到 mIndexMap 数组中，adjustIndex 的返回值决定音量是否继续往下设置，返回 ture 才网底层继续设置音量

```
3201                } else if (!isFullVolumeDevice(device)
3202                        && (streamState.adjustIndex(direction * step, device, caller,
3203                                hasModifyAudioSettings)
3204                                || streamState.mIsMuted)) {
3205                    // Post message to set system volume (it in turn will post a
3206                    // message to persist).
3207                    if (streamState.mIsMuted) {
3208                        // Unmute the stream if it was previously muted
3209                        if (direction == AudioManager.ADJUST_RAISE) {
```

静音状态下，音量加键解除静音

```
3210                            // unmute immediately for volume up
3211                            streamState.mute(false);
```

 版权所有 © 紫光展锐（上海）科技有限公司

重点注意 MSG_SET_DEVICE_VOLUME 消息

注意

MSG_SET_DEVICE_VOLUME 消息是实际实现往底层设置音量

```
                sendMsg(mAudioHandler,
                        MSG_SET_DEVICE_VOLUME,
                        SENDMSG_QUEUE,
                        device,
                        0,
                        streamState,
                        0);
            }

```

说明

这里看下 adjustIndex 这个方法，这个方法返回 true 才会往底层调用

>>>adjustIndex

```
    public boolean adjustIndex(int deltaIndex, int device, String caller,
                boolean hasModifyAudioSettings) {
```

调用 setIndex

```
            return setIndex(getIndex(device) + deltaIndex, device, caller,
                    hasModifyAudioSettings);
        }
```

Unisoc Confidential For longcheer

>>>setIndex 方法

```
        public boolean setIndex(int index, int device, String caller,
                boolean hasModifyAudioSettings) {
```

oldIndex 设备对应的当前音量

```
                    oldIndex = getIndex(device);
```

index 即将设置的音量

```
                    index = getValidIndex(index, hasModifyAudioSettings);
```

设备，音量值对应的键值对保存设备对应的音量值

```
                    mIndexMap.put(device, index);
```

音量是否发生了变化，最终 return 的就是 changed，即上一音量和即将调节的音量不相等才返回 true 继续向底层设置音量，这种情况发生在音量已经是最大了，继续调大音量就不再向底层设置音量了

```
                    changed = oldIndex != index;
```

 版权所有 © 紫光展锐（上海）科技有限公司

```
7610                if (changed) {

7621                    // fire changed intents for all streams
7622                    if (index != oldIndex) {
```

发送 VOLUME_CHANGED_ACTION 广播，通知音量变化，通知给到 systemui

```
7623                        mVolumeChanged.putExtra(AudioManager.EXTRA_VOLUME_STREAM_VALUE, index);
7624                        mVolumeChanged.putExtra(AudioManager.EXTRA_PREV_VOLUME_STREAM_VALUE, oldIndex);
7625                        mVolumeChanged.putExtra(AudioManager.EXTRA_VOLUME_STREAM_TYPE_ALIAS,
7626                                mStreamVolumeAlias[mStreamType]);
7627                        sendBroadcastToAll(mVolumeChanged);
7628                    }
7629                }
7630                return changed;
7631            }
```

说明

接下来继续看真正设置音量的 MSG_SET_DEVICE_VOLUME 消息

Unisoc Confidential For longcheer

>>>MSG_SET_DEVICE_VOLUME 消息

```
8039                    case MSG_SET_DEVICE_VOLUME:
8040                        setDeviceVolume((VolumeStreamState) msg.obj, msg.arg1);
```

>>>继续调用 setDeviceVolume

```
7938        /*package*/ void setDeviceVolume(VolumeStreamState streamState, int device) {
7939
7940            synchronized (VolumeStreamState.class) {
```

注意

重点注意 applyDeviceVolume_syncVSS 方法，这个方法是提交音量给底层进行音量设置，后面继续分析

```
7941                // Apply volume
7942                streamState.applyDeviceVolume_syncVSS(device);
```

提交映射此 stream 的其他流音量

```
7944                // Apply change to all streams using this one as alias
7945                int numStreamTypes = AudioSystem.getNumStreamTypes();
7946                for (int streamType = numStreamTypes - 1; streamType >= 0; streamType--) {
7947                    if (streamType != streamState.mStreamType &&
```

```
7948                            mStreamVolumeAlias[streamType] == streamState.mStreamType) {
7949                        // Make sure volume is also maxed out on A2DP device for aliased
stream
7950                        // that may have a different device selected
7951                        int streamDevice = getDeviceForStream(streamType);
7952                        if ((device != streamDevice)
7953                                && (isAbsoluteVolumeDevice(device)
7954                                || isA2dpAbsoluteVolumeDevice(device))) {
7955                            mStreamStates[streamType].applyDeviceVolume_syncVSS(device);
7956                        }
7957                        mStreamStates[streamType].applyDeviceVolume_syncVSS(streamDevice
);
7958                    }
7959                }
7960            }
```

通过 persistVolume 消息将音量信息保存到 setting 数据库，发送的 MSG_PERSIST_VOLUME 消息

```
7961            // Post a persist volume msg
7962            sendMsg(mAudioHandler,
7963                    MSG_PERSIST_VOLUME,
7964                    SENDMSG_QUEUE,
7965                    device,
7966                    0,
7967                    streamState,
7968                    PERSIST_DELAY);
7969
7970        }
```

Unisoc Confidential For longcheer

>>>MSG_PERSIST_VOLUME 消息就是把音量值写到了 setting 数据库

```
8047                    case MSG_PERSIST_VOLUME:
8048                        persistVolume((VolumeStreamState) msg.obj, msg.arg1);
8049                        break;
```

>>>persistVolume

```
7998            private void persistVolume(VolumeStreamState streamState, int device) {
```

将音量保存到 setting 数据库

```
8005                if (streamState.hasValidSettingsName()) {
8006                    mSettings.putSystemIntForUser(mContentResolver,
8007                            streamState.getSettingNameForDevice(device),
8008                            (streamState.getIndex(device) + 5)/ 10,
8009                            UserHandle.USER_CURRENT);
8010                }
8011            }
```

说明

继续看 applyDeviceVolume_syncVSS 提交音量到底层的方法

4.5 applyDeviceVolume_syncVSS

```
7507            /*package*/ void applyDeviceVolume_syncVSS(int device) {
```

…

继续调用 setStreamVolumeIndex

```
7520                setStreamVolumeIndex(index, device);
7521            }
```

>>>>setStreamVolumeIndex

```
7495            private void setStreamVolumeIndex(int index, int device) {
7496                // Only set audio policy BT SCO stream volume to 0 when the stream is actually muted.
7497                // This allows RX path muting by the audio HAL only when explicitly muted but not when
7498                // index is just set to 0 to repect BT requirements
7499                if (mStreamType == AudioSystem.STREAM_BLUETOOTH_SCO && index == 0
7500                        && !isFullyMuted()) {
7501                    index = 1;
7502                }
```

Unisoc Confidential For longcheer

通过这里调到 AudioSystem 中的 setStreamVolumeIndexAS 方法

```
7503                AudioSystem.setStreamVolumeIndexAS(mStreamType, index, device);
7504            }
```

4.6 setStreamVolumeIndexAS

frameworks/base/media/java/android/media/AudioSystem.java

这里通过 native 方法调用到底层去，传过去的参数是音频流类型，音量值，设备

这里的 DEBUG_VOLUME 默认是 false 关闭的，所以这里的 log 打不出来，出现调节音量问题，可以将这里的 log 打开，看下设置的 stream index device 三个参数是否正常，以及有没有调到这里来进一步排查是 java 层问题还是 native 层问题

```
1577        public static int setStreamVolumeIndexAS(int stream, int index, int device) {
```

 版权所有 © 紫光展锐（上海）科技有限公司

```
        if (DEBUG_VOLUME) {
            Log.i(TAG, "setStreamVolumeIndex: " + STREAM_NAMES[stream]
                    + " dev=" + Integer.toHexString(device) + " idx=" + index);
        }
        return setStreamVolumeIndex(stream, index, device);
    }
```

```
    private static native int setStreamVolumeIndex(int stream, int index, int device);
```

说明

以上部分就是按手机音量键调节音量 JAVA 部分的全部代码流程，主要由 PhoneWindow.java 接收到音量调节按键事件，经过 MediaSessionManager.java 和 MediaSessionService.java 的处理之后调用到 AudioManager.java 的 adjustSuggestedStreamVolumeForUid 方法，然后经过 AudioService.java 中的处理，最终通过 AudioSystem.java 的 setStreamVolumeIndexAS 方法调到 Native 层

Unisoc Confidential For longcheer

5 Native 层音量调节

接着继续分析 Native 层的音量调节过程

从 JNI 的 android_media_AudioSystem.cpp 的 android_media_AudioSystem_setStreamVolumeIndex 方法到 AudioPolicyManager.cpp 的 setStreamVolumeIndex 方法就是不断的处理调用过程，下面简单看下调用过程

5.1 JNI 中的 setStreamVolumeIndex

frameworks/base/core/jni/android_media_AudioSystem.cpp

```
android_media_AudioSystem_setStreamVolumeIndex(JNIEnv *env,
                                               jobject thiz,
                                               jint stream,
                                               jint index,
                                               jint device)
{
    return (jint) check_AudioSystem_Command(
```

Unisoc Confidential For longcheer

调用 AudioSystem.cpp 的 setStreamVolumeIndex 方法

```
            AudioSystem::setStreamVolumeIndex(static_cast<audio_stream_type_t>(stream),
                                              index,
                                              (audio_devices_t)device));
}
```

>>> AudioSystem.cpp 中的 setStreamVolumeIndex

frameworks/av/media/libaudioclient/AudioSystem.cpp

```
status_t AudioSystem::setStreamVolumeIndex(audio_stream_type_t stream,
                                           int index,
                                           audio_devices_t device) {
```

获取 audiopolicyservice 服务 aps

```
    const sp<IAudioPolicyService>& aps = AudioSystem::get_audio_policy_service();
```

调用到 aps 的 setStreamVolumeIndex 方法

 版权所有 © 紫光展锐（上海）科技有限公司

```
1180        return statusTFromBinderStatus(
1181                aps->setStreamVolumeIndex(streamAidl, deviceAidl, indexAidl));
```

接着调用的是 AudioPolicyInterfaceImpl.cpp 中的 AudioPolicyService::setStreamVolumeIndex 方法

>>>aps->setStreamVolumeIndex
frameworks/av/services/audiopolicy/service/AudioPolicyInterfaceImpl.cpp

```
1001   Status AudioPolicyService::setStreamVolumeIndex(AudioStreamType streamAidl,
1002                                                    const AudioDeviceDescription&
deviceAidl,
1003                                                    int32_t indexAidl) {
```

调用到 AudioPolicyManager 中的 setStreamVolumeIndex

```
1021        return binderStatusFromStatusT(mAudioPolicyManager->setStreamVolumeIndex(stream,
1022                                                                                 index,
1023                                                                                 device)
);
1024   }
```

说明

以上过程就是简单的调用过程，最终调用到 AudioPolicyManager 中开始进行音量的设置

Unisoc Confidential For longcheer

5.2 setStreamVolumeIndex

frameworks/av/services/audiopolicy/managerdefault/AudioPolicyManager.cpp

```
3011   status_t AudioPolicyManager::setStreamVolumeIndex(audio_stream_type_t stream,
3012                                                      int index,
3013                                                      audio_devices_t device)
3014   {
```

根据 stream 获取对应的 attributes 属性

```
3015        auto attributes = mEngine->getAttributesForStreamType(stream);
```

继续调用 setVolumeIndexForAttributes

```
3022        return setVolumeIndexForAttributes(attributes, index, device);
3023   }
```

>>> setVolumeIndexForAttributes

```
3039   status_t AudioPolicyManager::setVolumeIndexForAttributes(const audio_attributes_t
&attributes,
```

```
                                                                    int index,
                                                                    audio_devices_t device)
  {
```

根据 attributes 属性获取对应的音量组

```
      // Get Volume group matching the Audio Attributes
      auto group = mEngine->getVolumeGroupForAttributes(attributes);
```

根据 attributes 属性获取对应的音量曲线

```
      IVolumeCurves &curves = getVolumeCurves(attributes);
```

根据 group 转换对应的音量 VolumeSource

```
      VolumeSource vs = toVolumeSource(group);
```

根据 attributes 属性获取对应的音频策略

```
      product_strategy_t strategy =
mEngine->getProductStrategyForAttributes(attributes);

```

这里和 audioservice 中的 mIndexMap 有点像，不设置音量，但是是将音量值写到 curves 中的 map 数组 mIndexCur 中进行保存

```
      status = setVolumeCurveIndex(index, device, curves);
```

checkAndSetVolume 方法是真正进行音量设置的地方

```
          //FIXME: workaround for truncated touch sounds
          // delayed volume change for system stream to be removed when the problem is
          // handled by system UI
          status_t volStatus = checkAndSetVolume(
                      curves, vs, index, desc, curDevices,
                      ((vs == toVolumeSource(AUDIO_STREAM_SYSTEM, false))?
                          TOUCH_SOUND_FIXED_DELAY_MS : 0));
```

注意

checkAndSetVolume 方法是真正进行音量设置的地方

5.3 checkAndSetVolume

```
status_t AudioPolicyManager::checkAndSetVolume(IVolumeCurves &curves,
                                               VolumeSource volumeSource,
                                               int index,
                                               const sp<AudioOutputDescriptor>&
outputDesc,
                                               DeviceTypeSet deviceTypes,
                                               int delayMs,
                                               bool force)
```

Unisoc Confidential For longcheer

```
7373    {
```

这里计算对应的音量的 DB 值

```
7408        float volumeDb = computeVolume(curves, volumeSource, index, deviceTypes);
```

这里通过 outputDesc->setVolume 向下设

```
7416        outputDesc->setVolume(
7417                volumeDb, muted, volumeSource, curves.getStreamTypes(), deviceTypes,
delayMs, force);
```

如果是通话音量则调用 mpClientInterface->setVoiceVolume 向下设置音量

```
7430                mpClientInterface->setVoiceVolume(voiceVolume, delayMs);
7431                mLastVoiceVolume = voiceVolume;
7432            }
7433        }
7434        return NO_ERROR;
7435    }
```

说明

先看下非通话音量的音量设置过程 outputDesc->setVolume，主要就是各个部分的调用过程，最终到 audioflinger 中，通话音量设置过程 mpClientInterface->setVoiceVolume 后面再介绍

5.4 outputDesc->setVolume

Unisoc Confidential For longcheer

frameworks/av/services/audiopolicy/common/managerdefinitions/src/AudioOutputDescriptor.cpp

```
477    bool SwAudioOutputDescriptor::setVolume(float volumeDb, bool muted,
478                                            VolumeSource vs, const StreamTypeVector
&streamTypes,
479                                            const DeviceTypeSet& deviceTypes,
480                                            uint32_t delayMs,
481                                            bool force)
482    {
```

…

```
532        for (const auto &stream : streams) {
```

通过 mClientInterface->setStreamVolume 向下调用，这里的 ALOGV 默认不打印，音量问题调试时可以打出来看看

```
533            ALOGV("%s output %d for volumeSource %d, volume %f, delay %d stream=%s",
__func__,
534                  mIoHandle, vs, volumeDb, delayMs, toString(stream).c_str());
535            mClientInterface->setStreamVolume(stream, volumeAmpl, mIoHandle, delayMs);
536        }
```

```
537         return true;
538     }
```

mClientInterface->setStreamVolume 调用的是 AudioPolicyClientImpl.cpp 中的 setStreamVolume

>>>mClientInterface->setStreamVolume

frameworks/av/services/audiopolicy/service/AudioPolicyClientImpl.cpp

```
163   status_t AudioPolicyService::AudioPolicyClient::setStreamVolume(audio_stream_type_t stream,
164                                 float volume, audio_io_handle_t output,
165                                 int delay_ms)
166   {
```

调用 AudioPolicyService 的 setStreamVolume

```
167         return mAudioPolicyService->setStreamVolume(stream, volume, output,
168                                                     delay_ms);
169   }
```

>>>AudioPolicyService::setStreamVolume

frameworks/av/services/audiopolicy/service/AudioPolicyService.cpp

```
2809   int AudioPolicyService::setStreamVolume(audio_stream_type_t stream,
2810                                           float volume,
2811                                           audio_io_handle_t output,
2812                                           int delayMs)
2813   {
2814       return (int)mAudioCommandThread->volumeCommand(stream, volume,
2815                                                      output, delayMs);
2816   }
```

```
2325   status_t AudioPolicyService::AudioCommandThread::volumeCommand(audio_stream_type_t stream,
2326                                                                  float volume,
2327                                                                  audio_io_handle_t output,
2328                                                                  int delayMs)
2329   {
2330       sp<AudioCommand> command = new AudioCommand();
2331       command->mCommand = SET_VOLUME;
```

发送 SET_VOLUME 指令

```
2340       return sendCommand(command, delayMs);
2341   }
```

Unisoc Confidential For longcheer

```
2001  bool AudioPolicyService::AudioCommandThread::threadLoop()
2002  {
…
2010          while (!mAudioCommands.isEmpty() && !exitPending()) {
…
```

通过这个 threadloop 继续向下调用到 AudioSystem 的 setStreamVolume 方法

```
2022                  case SET_VOLUME: {
2023                      VolumeData *data = (VolumeData *)command->mParam.get();
2024                      ALOGV("AudioCommandThread() processing set volume stream %d, \
2025                              volume %f, output %d", data->mStream, data->mVolume,
data->mIO);
2026                      mLock.unlock();
2027                      command->mStatus = AudioSystem::setStreamVolume(data->mStream,
2028                                                                      data->mVolume,
2029                                                                      data->mIO);
2030                      mLock.lock();
2031                      }break;
```

>>>AudioSystem::setStreamVolume

frameworks/av/media/libaudioclient/AudioSystem.cpp

```
212  status_t AudioSystem::setStreamVolume(audio_stream_type_t stream, float value,
213                                        audio_io_handle_t output) {
214      if (uint32_t(stream) >= AUDIO_STREAM_CNT) return BAD_VALUE;
```

获取 AudioFlinger 服务 af

```
215      const sp<IAudioFlinger>& af = AudioSystem::get_audio_flinger();
216      if (af == 0) return PERMISSION_DENIED;
```

调用 af 的 setStreamVolume 方法

```
217      af->setStreamVolume(stream, value, output);
218      return NO_ERROR;
219  }
```

>>>AudioFlinger::setStreamVolume

frameworks/av/services/audioflinger/AudioFlinger.cpp

```
1623  status_t AudioFlinger::setStreamVolume(audio_stream_type_t stream, float value,
1624          audio_io_handle_t output)
1625  {
1642      VolumeInterface *volumeInterface = getVolumeInterface_l(output);
```

Unisoc Confidential For longcheer

如果有 active 的播放线程，这里调用到 Threads.cpp 的 setStreamVolume

```
    volumeInterface->setStreamVolume(stream, value);

    return NO_ERROR;
}
```

frameworks/av/services/audioflinger/Threads.cpp

```
void AudioFlinger::PlaybackThread::setStreamVolume(audio_stream_type_t stream, float value)
{
    Mutex::Autolock _l(mLock);
    mStreamTypes[stream].volume = value;
    broadcast_l();
}
```

说明

至此非通话音量的设置就结束了，可以看到非通话音量的设置没有经过 hal，在 FW 层就设置完毕了

Unisoc Confidential For longcheer

5.5 AudioPolicyClient::setVoiceVolume

下面看下通话音量的调整

frameworks/av/services/audiopolicy/service/AudioPolicyClientImpl.cpp

```
status_t AudioPolicyService::AudioPolicyClient::setVoiceVolume(float volume, int delay_ms)
{
    return mAudioPolicyService->setVoiceVolume(volume, delay_ms);
}
```

>>>mAudioPolicyService->setVoiceVolume
frameworks/av/services/audiopolicy/service/AudioPolicyService.cpp

```
int AudioPolicyService::setVoiceVolume(float volume, int delayMs)
{
    return (int)mAudioCommandThread->voiceVolumeCommand(volume, delayMs);
}
```

 版权所有 © 紫光展锐（上海）科技有限公司

```
status_t AudioPolicyService::AudioCommandThread::voiceVolumeCommand(float volume, int delayMs)
{
    sp<AudioCommand> command = new AudioCommand();
    command->mCommand = SET_VOICE_VOLUME;
```

SET_VOICE_VOLUME 指令

```
    return sendCommand(command, delayMs);
}
```

>>>SET_VOICE_VOLUME

```
                case SET_VOICE_VOLUME: {
                    VoiceVolumeData *data = (VoiceVolumeData *)command->mParam.get();
                    ALOGV("AudioCommandThread() processing set voice volume volume %f",
                            data->mVolume);
                    mLock.unlock();
```

调用到 AudioSystem::setVoiceVolume

```
                    command->mStatus = AudioSystem::setVoiceVolume(data->mVolume);
                    mLock.lock();
                    }break;
```

Unisoc Confidential For longcheer

>>>AudioSystem::setVoiceVolume

frameworks/av/media/libaudioclient/AudioSystem.cpp

```
status_t AudioSystem::setVoiceVolume(float value) {
    const sp<IAudioFlinger>& af = AudioSystem::get_audio_flinger();
    if (af == 0) return PERMISSION_DENIED;
```

调用 AudioFlinger 的 setVoiceVolume 方法

```
    return af->setVoiceVolume(value);
```

>>> AudioFlinger::setVoiceVolume

```
status_t AudioFlinger::setVoiceVolume(float value)
{
```

从这里开始调到 hal 去设置通话音量,FW 层的设置到此结束，hal 层的流程这里不再继续分析

```
    sp<DeviceHalInterface> dev = mPrimaryHardwareDev->hwDevice();
    mHardwareStatus = AUDIO_HW_SET_VOICE_VOLUME;
    ret = dev->setVoiceVolume(value)
```

 版权所有 © 紫光展锐（上海）科技有限公司

A 附录（Appendix heading 1）

A.1 代码（Appendix heading 2）

Android T Audio FW 相关代码

A.2 其它参考信息

FW audio 介绍.docx

Framework 层音量设置流程代码分析.docx

【多媒体】A12 音量键调节音量流程.docx

Unisoc Confidential For longcheer